OpenRoads SDK

Unlocking your superpowers

23rd April 2026

AtkinsRéalis

Introduction

Who I am:

- Chris Andrew – Global ProjectWise Lead in AtkinsRéalis IT Design and Engineering Solutions Team.
- 26+ years in architecture, civil, digital engineering & information management
- OpenRoads Premier Scholar (since 2021 EMEA)
- I am a Gamer, and when I was younger (Half-life), I used to Mod (sometime cheat) video game by what I know understand as coding.
- I’m a father to 2 boys under 5
- I have 2 Huskies
- I’m a footballer (its not called soccer! Lol) and a snowboarder.

Why I’m here

- I build production tools, to help make life easier for delivery teams
- Automation-first, standards-driven, governance-aware
- Strong focus on SDK + ProjectWise + real project delivery

I’m not a software developer by training or trade.

Session Aims and Audience

Who is this for?

- Premier Scholars,
- engineers,
- digital specialists, and
- practitioners seeking to amplify their impact with OpenRoads SDK.

Session Aim:

Demystify the OpenRoads SDK, show what is realistically achievable, and inspire you to explore new ways to automate, integrate, and innovate within civil design workflows.

What is OpenRoads SDK

Core idea

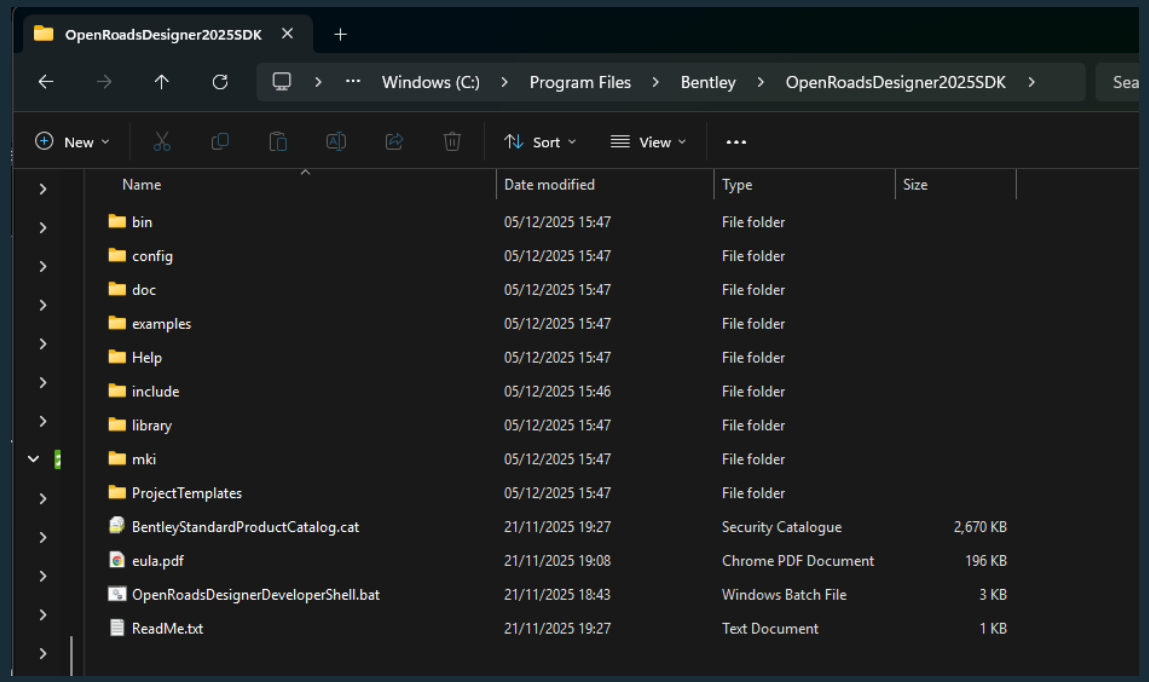
- The OpenRoads SDK is Bentley's software development kit for extending OpenRoads Designer, OpenRail Designer and ProjectWise Integration
- It enables custom tools, automation, integrations, and domain-specific workflows directly inside ORD, ORLD and PW.

What it gives you

- Direct access to Civil geometry, models, civil objects, features, and metadata
- Ability to build:
  - Native add-ins (C# / C++)
  - Custom commands and tools

Why it matters

- Turns OpenRoads from a *tool you use* into a *platform you build on*
- Allows engineers to remove repetitive work, enforce standards, and integrate with wider digital ecosystems

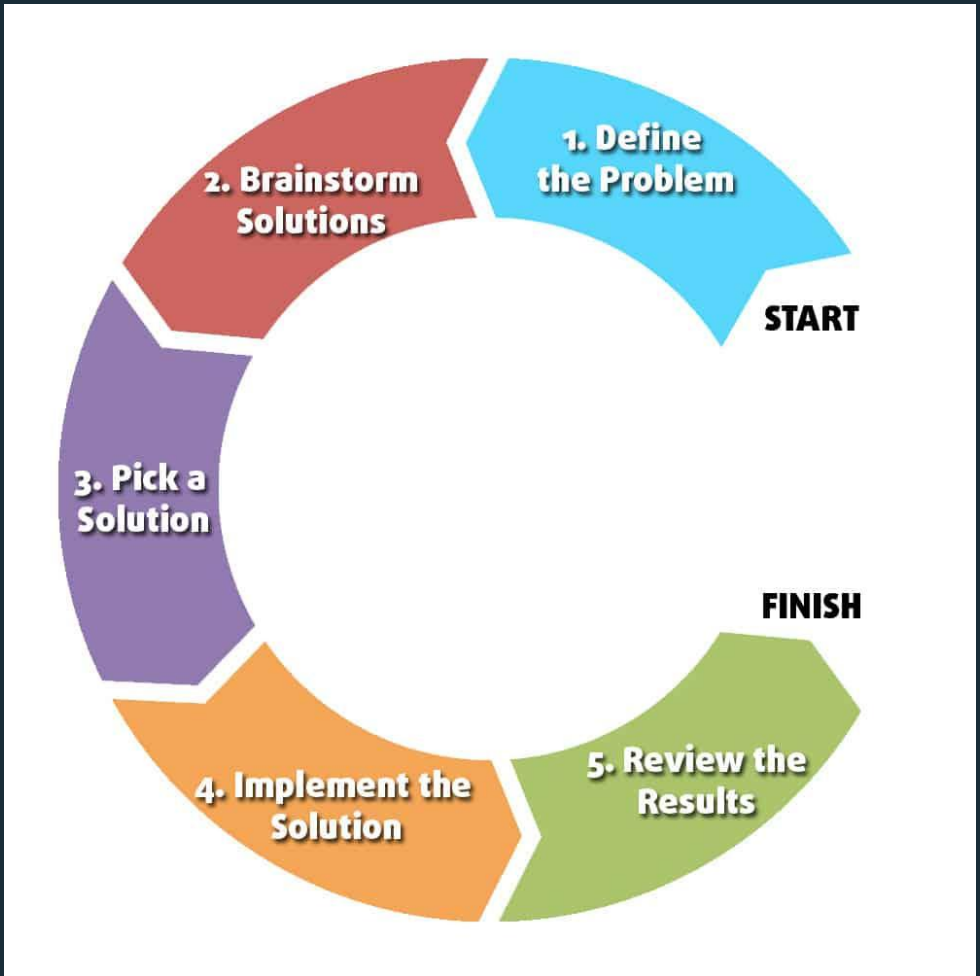


Why Explore the SDK?

Explore the SDK when you need to:

- Move beyond out-of-the-box functionality
  - tailor OpenRoads to your project,
  - workflow, and
  - data integration needs.
- Unlock new superpowers:
  - automation,
  - integration, and
  - custom solutions.

The SDK is powerful but assumes you know what you're doing—this session aims to bridge that gap.



Where to get it?

SDK availability

- The OpenRoads Designer SDK is **not bundled** with the standard product install
- It can be downloaded separately via Bentley's **Software Downloads**

Prerequisites

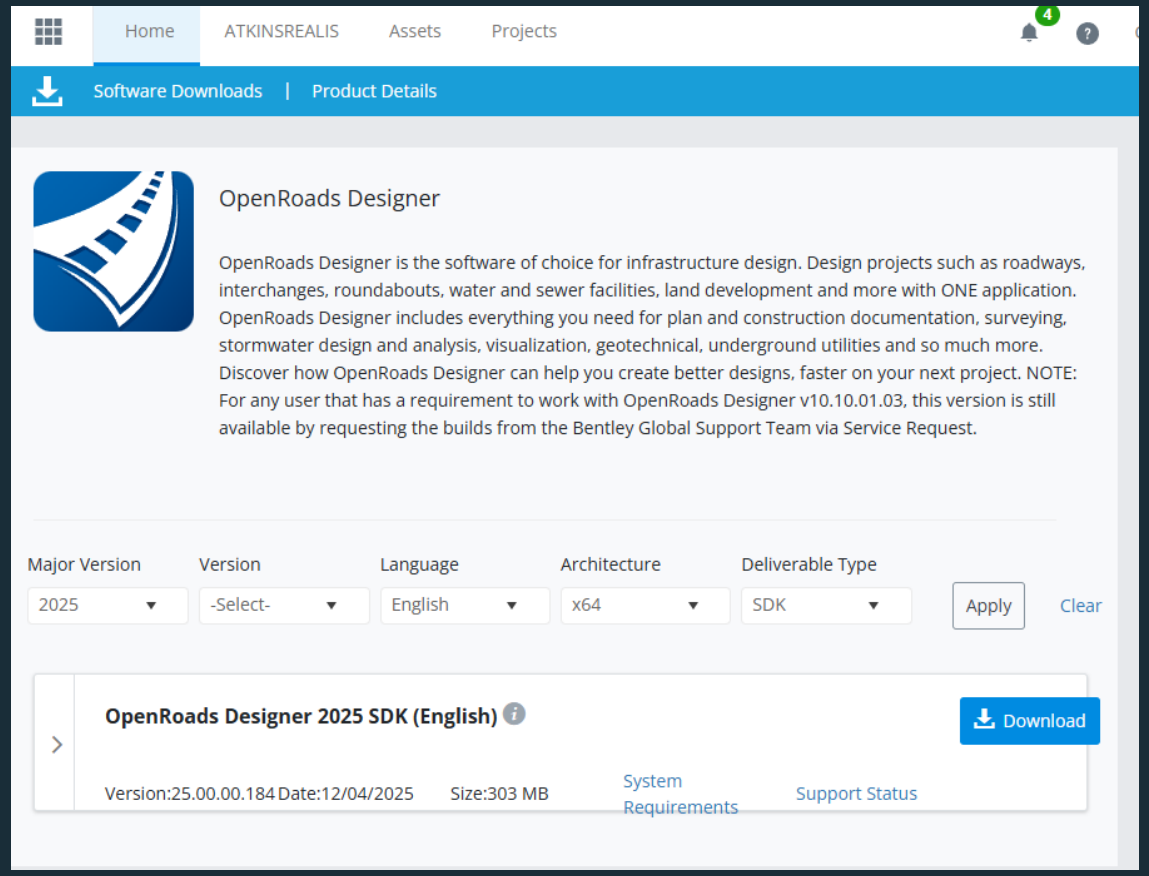
- Installed version of OpenRoads Designer
- Bentley Developer Network (BDN) access
  - Typically via:
    - In-house development entitlement
    - Approved developer access within your organization

What you install

- SDK binaries matched to your OpenRoads version
- Developer tools and headers
- Example projects and utilities

Key takeaway: **SDK version alignment with ORD is critical**

Software Downloads



Where to find documentation?

Primary sources

- Bentley online SDK documentation (API reference & guides)
- Developer Guide (managed & unmanaged SDK)
- Example projects shipped with the SDK

What the docs are good at

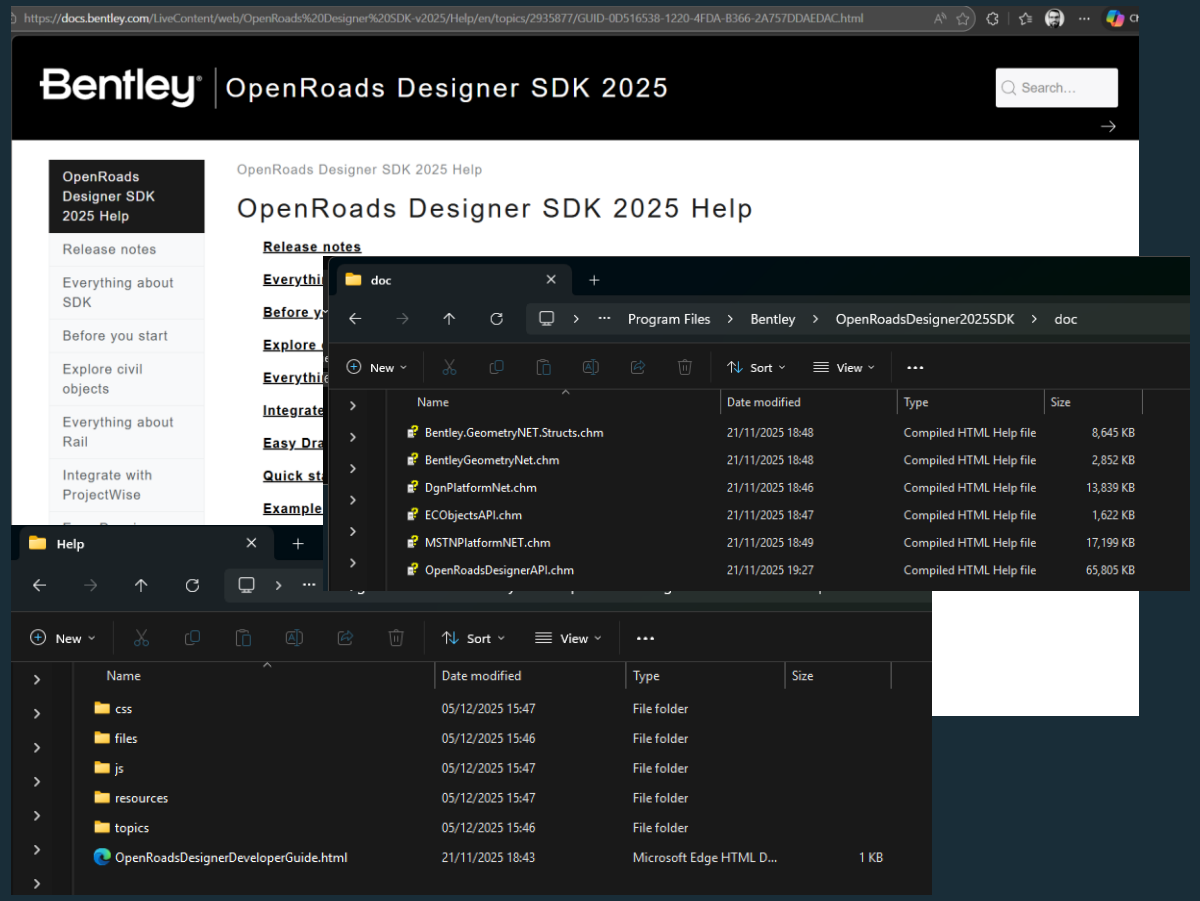
- Explaining object models and namespaces
- Showing *what* is possible
- Providing reference-level API detail

What they're less good at

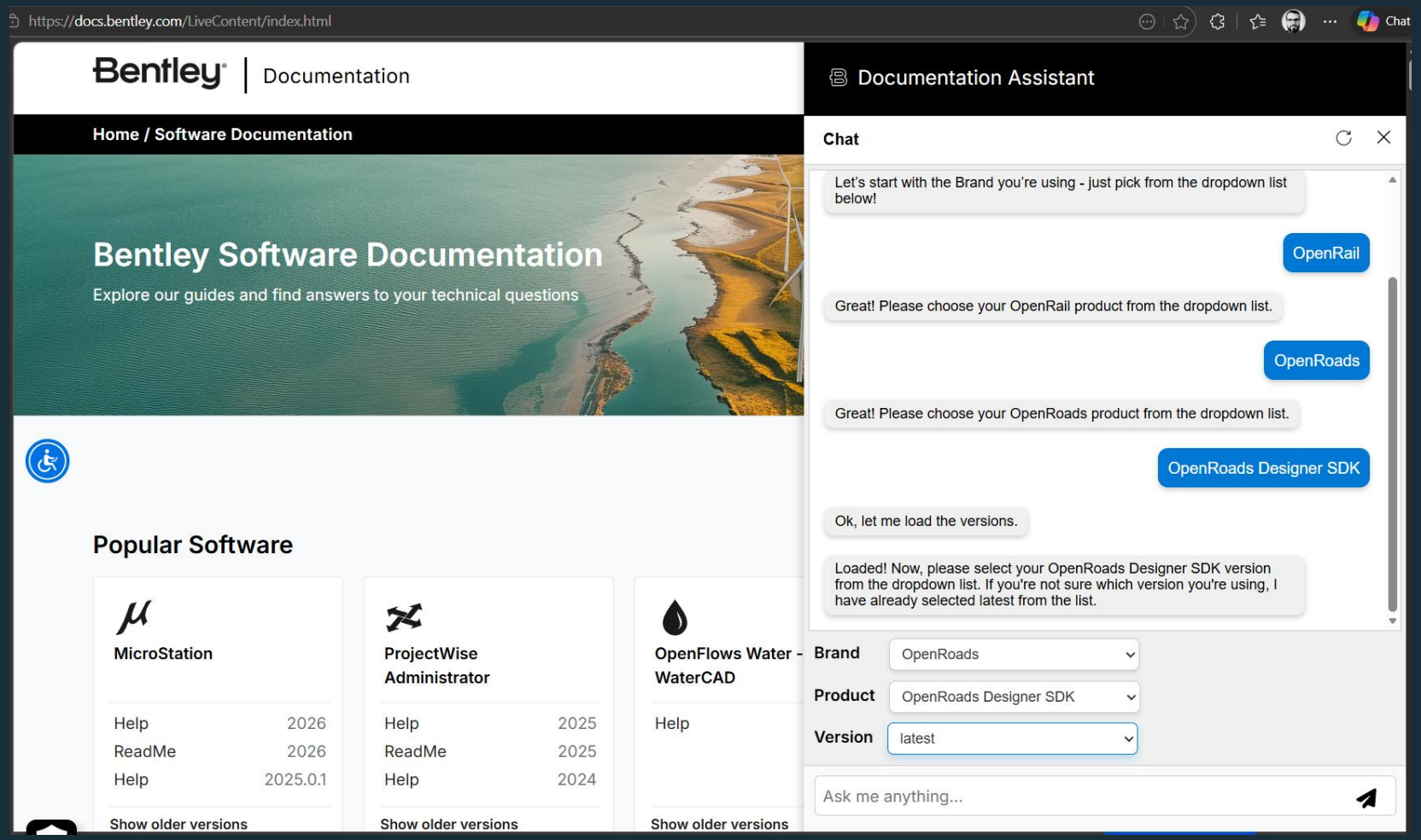
- End-to-end real-world workflows
- Integration patterns across systems
- Performance and scalability considerations

This is where **experimentation and real projects** become essential

OpenRoads Designer SDK 2025 Help



Where to find documentation?



What’s included?

SDK components

- Managed SDK (C# / .NET)
- Unmanaged SDK (C++)
- Geometry, civil, terrain, and model APIs
- ProjectWise integration points (contextual)

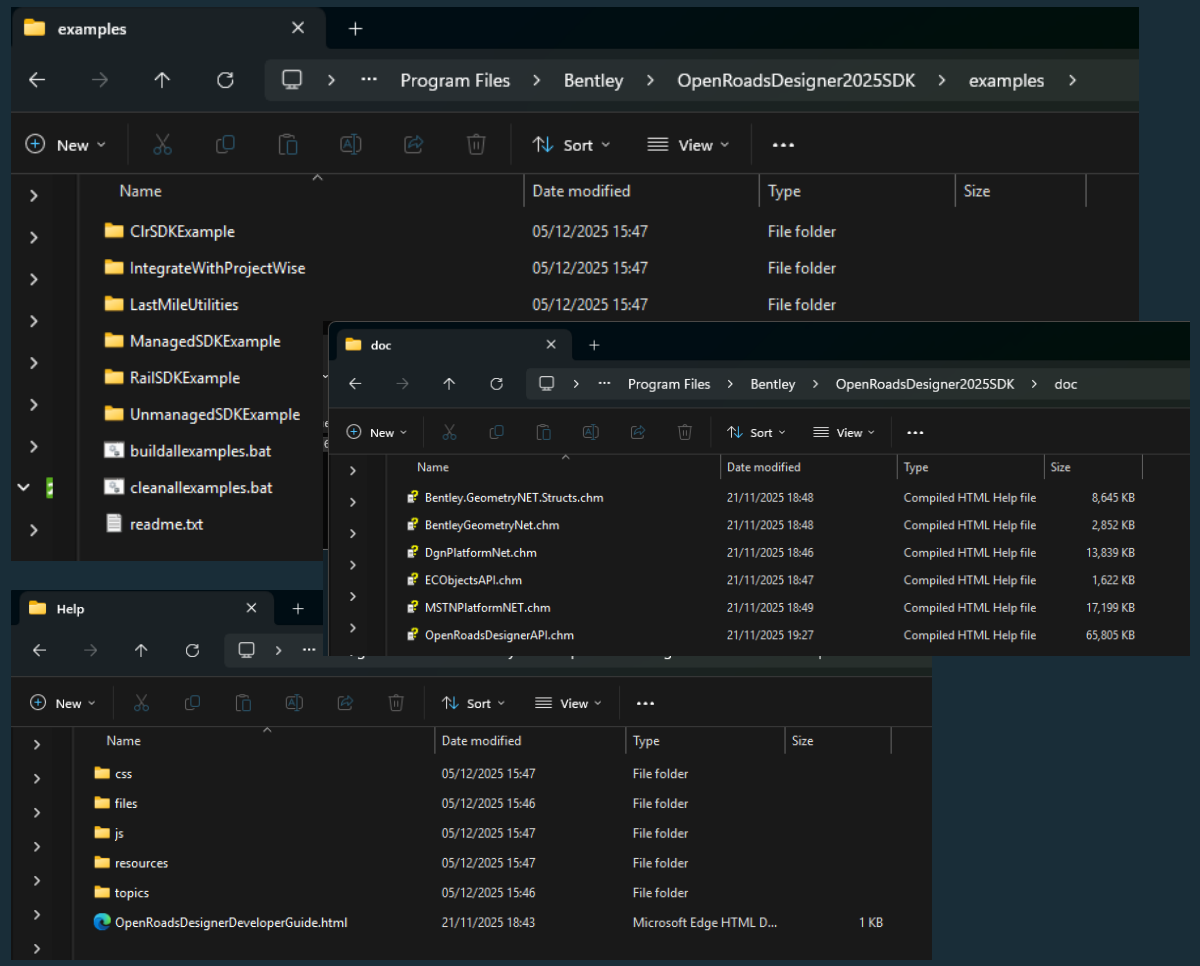
Development support

- Sample code and utilities
- Developer command shell
- Debugging support within Visual Studio

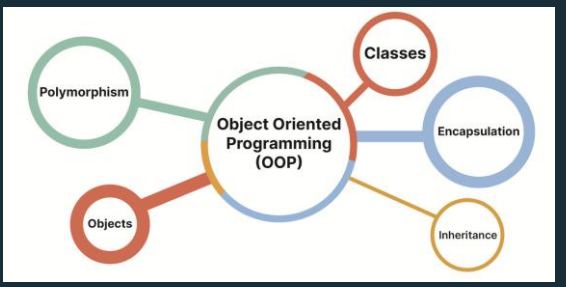
What you don’t get

- Turnkey solutions
- Opinionated architecture
- Guardrails on performance or misuse
- Generative Components API (separate)

The SDK is powerful — **but it assumes you know what you’re doing**



How to get started?

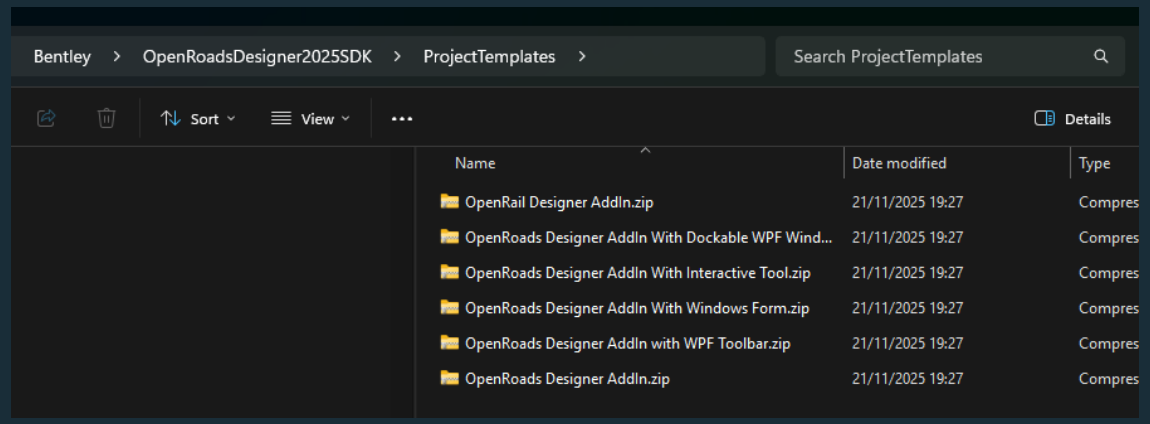


Recommended approach

1. Learn (at least the basics of) C#
2. Learn Basics of Object Orientated Programming

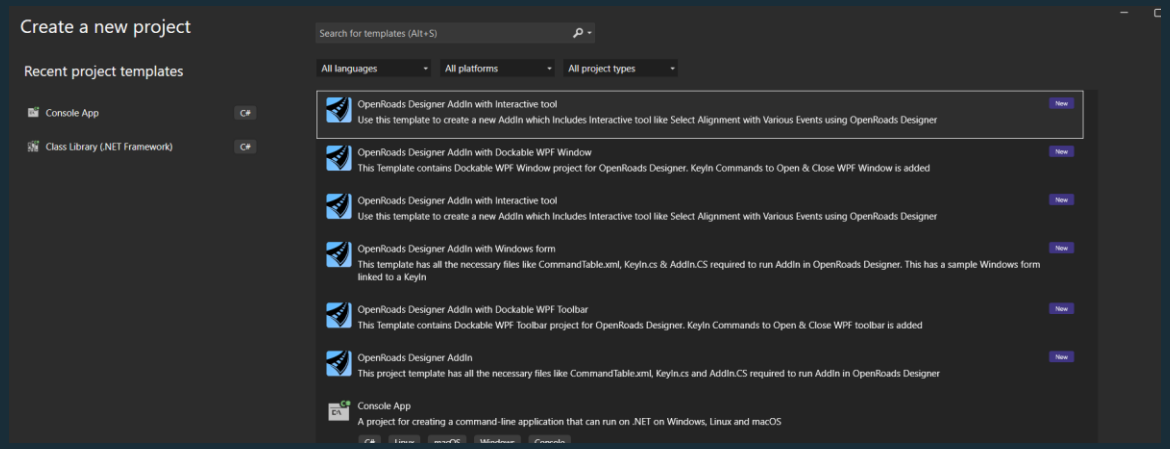
Then:

3. Start with a **small, well-defined problem**
4. **Use a Project Template** provided in the SDK
5. Use the managed SDK unless you have a strong reason not to
6. Learn the OpenRoads object model before writing large amounts of code



Typical first steps

- Build a simple add-in that:
  - Reads model data
  - Creates or modifies elements
  - Writes structured output (JSON, CSV, logs)
- Debug directly inside OpenRoads Designer



Key mindset shift

- You are now developing *inside* a live design environment
- Performance, stability, and user experience matter

What I've achieved

This section shows **real examples** of how the SDK has been used in production-style scenarios — not demos.

Focus areas:

- Integration
- Automation
- Data-driven workflows
- Parametric extension

What I've achieved

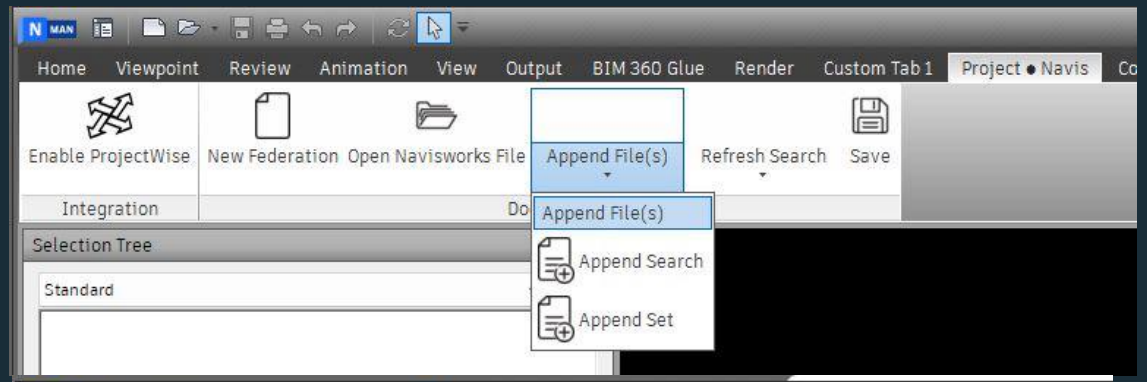
Navisworks and ProjectWise Integration

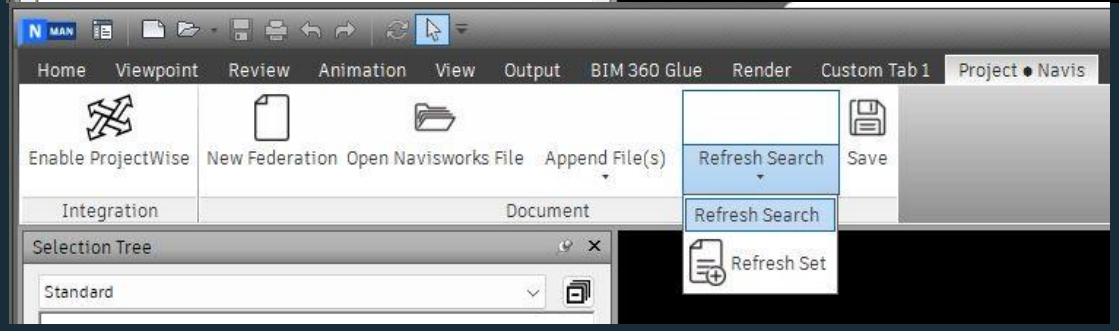
Problem Statement

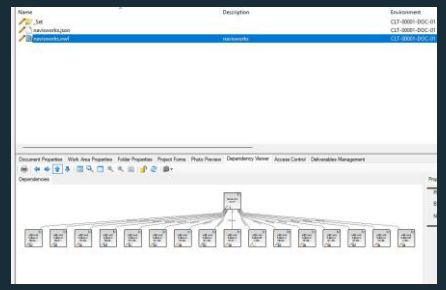
- Navisworks used for coordination & design review
- Source models governed in ProjectWise
- Typical workflows rely on:
  - Exported copies
  - Manual file handling
  - Version ambiguity
- **Result**: lost auditability, duplicated data, weak information management

Solution: ProjectNavis-Link

- Lightweight Navisworks plugin
- Browse ProjectWise content **inside Navisworks**
- Open managed models **without unmanaged local copies**
- Enables coordination while:
  - Preserving version control
  - Maintaining provenance
  - Keeping ProjectWise as the source of truth







What I've achieved

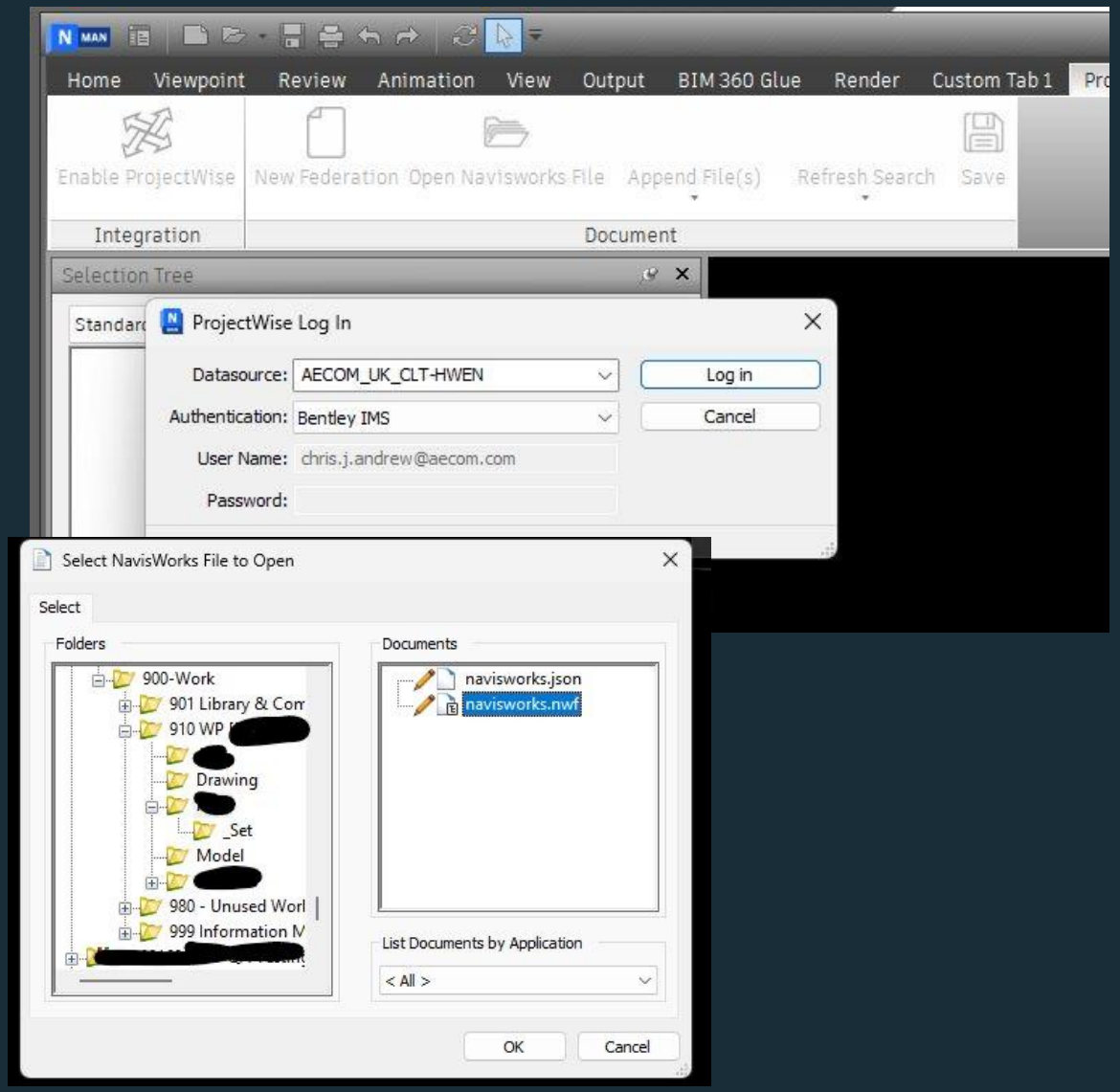
Navisworks and ProjectWise Integration

Key Capabilities

- Authenticates with ProjectWise services
- Enumerates ProjectWise folders in Navisworks
- Opens managed models directly into review workflows
- Uses standard Navisworks tools:
  - Clash detection
  - Mark-up
  - Viewpoints
- **Does not bypass governance or lifecycle controls**

Why the SDK

- Not solvable with configuration or scripting alone
- Required:
  - Native ProjectWise API access
  - In-application Navisworks integration
  - Controlled handling of managed files
- **SDK enables workflow continuity, not just automation**

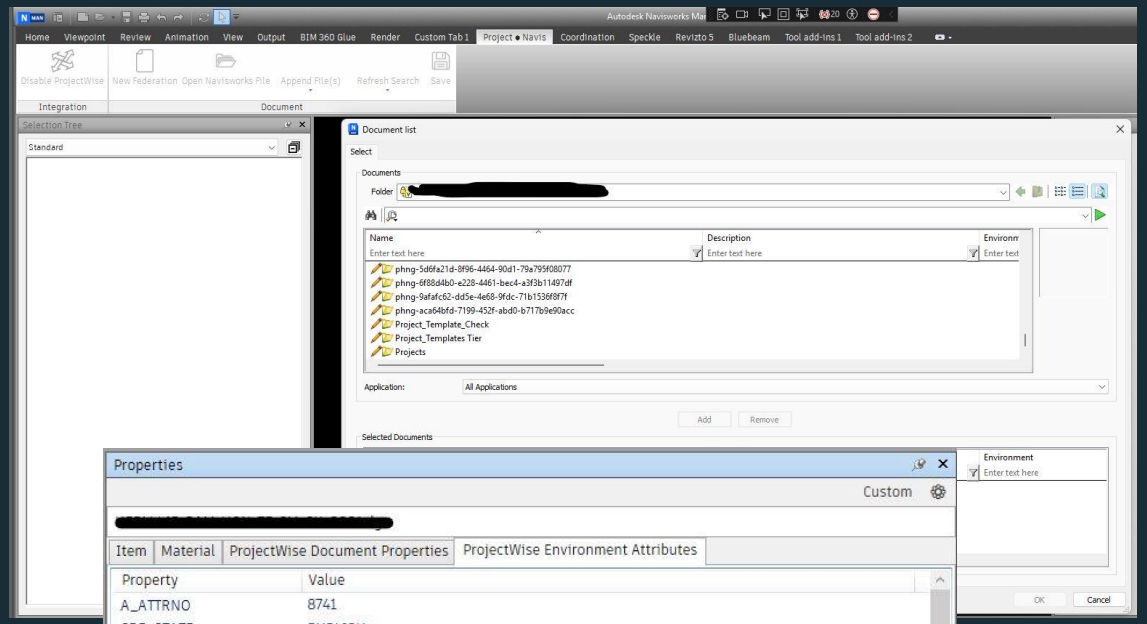


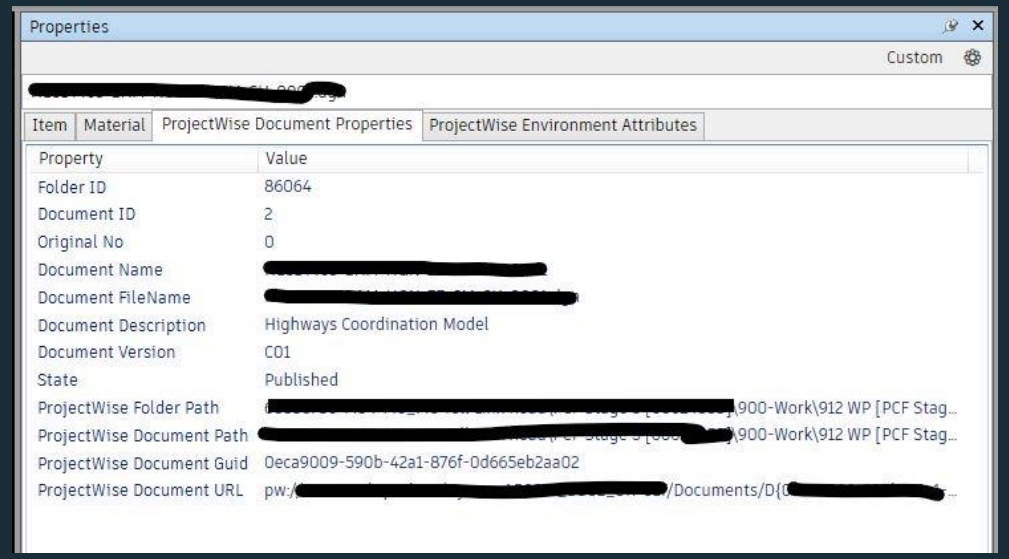
What I've achieved

Navisworks and ProjectWise Integration

What This Demonstrates

- SDKs are justified when:
  - Governance must be preserved end-to-end
  - Review tools sit outside authoring platforms
  - Manual exports break auditability
- SDK acts as an **integration layer**, not a geometry tool





What I've achieved

Data Extraction, Validation and Import

Problem Statement

- Disconnected design, coordination, and information management workflows

Solution: {ARES}:DataExchange

- Custom tooling using the OpenRoads SDK to:
  - Extract structured design and metadata
  - Link OpenRoads outputs to ProjectWise-managed data
  - Data Exchange between Engineering authoring tools
  - Allowed a backup of the Civil content, re-import if failure happens
  - Alternative to IFC, LandXML, Speckle

Outcome

- Reduced manual handoffs
- Consistent data lineage
- Better coordination without rework

What I've achieved

High-volume event point creation

Problem Statement

- Civil workflows rely on **event points along alignments**
- Out-of-the-box ORD tools:
  - Don't scale to high-volume or bespoke workflows
  - Don't expose clean access to alignment geometry or metadata
- Configuration, VBA, or macros cannot:
  - Participate in ORD's interactive tool lifecycle
  - Reliably access CIF (Consensus Infrastructure Framework) data
- **Result**: a platform-level limitation, not a UI or scripting gap

Solution: EventPoints

- C# OpenRoads Designer add-in (ORD 25)
- Hooks directly into:
  - ORD command and tool infrastructure
  - Interactive DGN workflows
- Enables:
  - Interactive alignment selection
  - Direct access to alignment geometry and metadata
- Designed as:
  - A reusable foundation
  - Not a one-off tool

Key Capabilities

- Registers native ORD key-ins and commands
- Implements a custom interactive selection tool
- Resolves and validates:
  - Alignments
  - Profiles
  - Geometry via the Consensus Connection
- Fully integrated with ORD runtime:
  - No external executables
  - No detached processing

Why the SDK

- Required access to:
  - Geometric Model & alignment APIs
  - ORD interactive tool lifecycle
  - CIF alignments and profiles
- Not achievable with:
  - Workspace configuration
  - VBA
  - One-off scripts
- **SDK enables deep platform integration—not surface automation**

What This Demonstrates

- SDK use is justified when you need to:
  - Interact natively with ORD geometry
  - Build reusable tools, not disposable scripts
  - Extend ORD behaviour in a supported, controlled way
- **This is building capability into the platform**

What I've achieved

Custom Generative Component Nodes

Problem Statement

- Native ORD drawing production has structural limitations:
  - Section creation from named boundaries is slow
  - Dynamic graphics, static annotation
  - Fragile DWG export workflows
  - Corridor changes force repeated rework
- **Core issue**: lack of repeatability and change resilience
- The problem wasn't geometry — it was process durability

Solution: {GenDes}:OpenRoads

- Generative Components add-on for OpenRoads Designer 2025
- Provides a **parametric, graph-driven pipeline** for:
  - Alignment & corridor resolution
  - Cross-section extraction
  - Annotation band generation
  - Sheet layout
- One graph = **single source of intent**
- Change inputs → everything downstream regenerates

Key Capabilities

- Resolve alignments & corridors once, reuse everywhere
- Bulk cross-section extraction:
  - Cut / fill
  - Volumes
  - Existing ground sampling
- Generate clean 2D section geometry for sheets
- Parametric annotation bands & layouts
- Integrated data tooling:
  - Excel read/write
  - JSON query
  - IFC export
  - Item Types read/write (Thanks to Ed Ashbolt for this)
- Modular node sets — use only what you need

Why the SDK

- Native tools don't expose:
  - Cross-section data pipelines
  - Annotation generation logic
  - Robust data I/O inside ORD
- GC scripting alone is insufficient without:
  - CIF & geometry access
  - Item Types
  - Platform-safe data exchange
- **SDK enables platform-level parametrics, not just visual scripting**

What This Demonstrates

- SDK use is justified when you need:
  - Deterministic, repeatable outputs
  - Bulk operations over station-by-station workflows
  - Change-driven regeneration, not redraws
- **This is design intent encoded into the workflow**

What I've achieved

Custom Generative Component Nodes

Also developed for GC:

- An Custom built Excel Node, as Bentley built one uses Excel COM objects and this has broken now AtkinsRealis uses Sensitivity Labels (Microsoft Purview)
- JSON Import / Export
- IFC Exporter
- Expanded Ed Ashbolts GC Communities Item Type Node

Demonstration

Dev, Build, Deploy

AtkinsRéalis

When and When Not

Use the SDK when:

You have an **immediate requirement that the core product does not support**

A workflow issue **cannot be solved with configuration, standards, or existing tools**

You are **integrating with systems outside Bentley's ecosystem**

e.g. ProjectWise, Navisworks, Excel, JSON, IFC, external platforms

Waiting for core product development is **not viable for project delivery**

You need **repeatability, scale, or automation**, not a one-off workaround

The capability needs to live **inside OpenRoads Designer**, not alongside it

Do not use the SDK when:

Your business **cannot support custom add-in ownership and maintenance**

You are trying to **work around or "fix" a core product defect**

The problem should clearly be addressed through **product enhancement**, not custom code

Configuration, standards, or training would solve the issue just as effectively

You are unwilling or unable to **support, version, and maintain the add-in**

You expect the SDK to provide **short-term relief without long-term responsibility**

The SDK isn't a shortcut – it's a commitment.
Use it when the value clearly outweighs the responsibility.

What next?

Ownership, Deployment, maintenance and survivability

The SDK lets you build powerful solutions —

but ownership, deployment, and support determine whether they survive.

1. Ownership & Intellectual Property

Be explicit about ownership up front

If you build the solution using:

- Company licenses
- A company laptop
- Company GitHub / repos
- Company time
  → **the business owns the IP**

Don't leave this ambiguous — it creates problems later

2. Deployment Strategy

Decide **how the solution reaches end users**

Preferred approach:

- **IT-managed installation** (controlled, auditable, supportable)
- Viable alternative:
- **ProjectWise Managed Workspaces**
- Effective for deploying add-ins and customisations
- Versioned, governed, and scalable

3. Support & Ongoing Maintenance

A working solution is not a finished solution

Be clear on:

- **Who supports it**
- **Who pays for upkeep**
- If you personally own it:
  - Have a contingency plan for support handover
- If the business owns it:
  - Make sure maintenance is explicitly funded

4. Versioning & Longevity

Assume the tool will be used **for more than one ORD release**

Plan for:

- Supporting multiple OpenRoads versions
- Testing against new releases
- Updating as APIs evolve
- If you don't plan for upgrades, the tool will die silently

5. Documentation & Training

Documentation is not optional

At minimum:

- What the tool does
- How it's installed
- How it's used
- How it's supported

Training matters:

- Even the best tool fails without adoption
- Short guides beat clever code

Plan

↓

Develop → Test → Repeat x n

↓

Document

↓

Deliver

Questions

AtkinsRéalis

Useful links

OpenRoads Designer SDK

- **SDK Help & API Reference (ORD 2025)**
  https://docs.bentley.com/LiveContent/web/OpenRoads%20Designer%20SDK-v2025/Help/en/topics/2935877/GUID-0D516538-1220-4FDA-B366-2A757DDAEDAC.html
- **SDK Installation & Setup**
  https://docs.bentley.com/LiveContent/web/OpenRoads%20Designer%20SDK%20Help-v1/en/GUID-FD0E7B04-66C7-4576-8CB7-95E8CABDB8A8.html
- **Developer Guide & Example Projects**
  https://bentleysystems.service-now.com/community?id=kb_article_view&sysparm_article=KB0012549
- **Civil Programming Forum (SDK questions & answers)**
  https://communities.bentley.com/products/programming/civil_programming/f/civil_programming_forum

C# & .NET Framework (ORD-relevant)

- **.NET Framework 4.8 Overview**
  https://learn.microsoft.com/en-gb/dotnet/framework/get-started/overview
- **.NET Framework Class Library Reference**
  https://learn.microsoft.com/en-gb/dotnet/framework/reference/
- **C# Language Reference**
  https://learn.microsoft.com/en-gb/dotnet/csharp/language-reference/
- **Object-Oriented Programming in C#**
  https://learn.microsoft.com/en-gb/dotnet/csharp/fundamentals/object-oriented/

Generative Components (GC + SDK)

- **Generative Components Documentation**
  https://docs.bentley.com/en/GenerativeComponents/index.php
- **Generative Components API (Add-ins & Custom Nodes – PDF)**
  https://docs.bentley.com/LiveContent/web/GenerativeComponentsAPI-En_PDF-v1/en/GenerativeComponents-API.pdf
- **GC Learning Resources & Playlists**
  https://bentleysystems.service-now.com/community?id=kb_article_view&sysparm_article=KB0097568

GitHub

- https://github.com/CJAndrew84/VSToolsForMicroStationCONNECTEdition
- CJAndrew84/PremierScholar_ORD_SDK
- edashbolt/generative-components: Custom Generative Components Nodes

thank you

AtkinsRéalis